# Large-Scale Knowledge Graph Representation for Personal Health Records

Master Research - Project Report

*Authors*
Mathias Yap i6192890, Vincent Hoogstraten i6372959, Johannes Schillberg i6407958, Annamaria Grillhoesl i6026290, Smit Vaidya i6350809

*Supervisors*
Dr Remzi Celebi, Shervin Mehryar

*Coordinator*
Dr Linda Rieswijk

June 11, 2025

## 1 Introduction

The European Commission aims for all EU-citizens to have full access to their electronic health records (EHR) by 2030 [1]. Enabling this access would allow citizens to get control of their own health data, meaning they can play an active role in managing it, make informed decisions regarding treatment options and be involved in preventive care considerations. Furthermore, it supports healthcare professionals to make better diagnoses. Lastly, it helps researchers by providing access to larger volumes of better-integrated datasets [2]. In order to fulfill this mission, electronic health records must be provided in a unified format, with semantically aligned concepts and support for machine-to-machine interoperability [3]. Additionally, patient records change over time, as they capture multiple visits and interactions over time with general practitioners and other healthcare professionals. Therefore, in order to accurately reflect the progression of a patients condition, the data must be able to track different activities for the same patient. These data prerequisites make the design and implementation of information systems that are usable and effective a significant challenge.

As various independent research efforts have shown, a promising candidate for addressing these challenges involves using representations based on the Resource Description Framework (RDF) standard, such as the Web Ontology Language (OWL) [4]. A natural implementation choice of RDF based concepts is the use of Knowledge Graphs. While these representations address challenges related to data collection, integration and unification, the task of retrieving knowledge within that framework brings its own challenges. Ideally, personal health data should be reusable, consistent, AI-ready, and interoperable across systems. A promising approach is the integration of data from multiple sources into a unified, large-scale personal health knowledge graph. Such a graph not only enables advanced querying capabilities, but also facilitates the generation of meaningful embeddings that can be leveraged in machine learning tasks [5].

One critical factor to the successful adoption of these systems, is their usability and comprehensibility. Health records stored in formal data structures such as graph databases are not directly accessible to individuals without training in data querying languages such as SQL or SPARQL and may provide information that is irrelevant to the user in question. For example, would a medical professional running a clinical trial need to know how the sensitive information of a patient is encoded to run their clinical trial? In order to address this, a user-friendly framework that adapts to the needs of the user and provides them with pertinent information is necessary. Part of our work will therefore be on building user profiles for users that are most likely to use the system. To do this we need a clear understanding of the order of how such a system would function.

## 1.1 Objective

The goal of this project is (1) to generate a representative set of user question–query pairs tailored to a specific user profile, and (2) to generate optimal queries from user questions that can be executed directly on the graph. Optimal queries are those that accurately retrieve the intended results based on the user question, while minimizing execution time.

## 1.2 Background - Clinical Research

Clinical research is medical research that involves human participants [6]. In clinical research, the effectiveness of drugs on patients is assessed using various methodologies, one of which is the clinical trial [7]. To prepare, conduct, evaluate and post-process clinical trials ethically and in accordance with research standards, a wide range of information needs must be addressed. This, together with the fact that a clinical trial has well established standards, makes it a good example to showcase our methodology. Our goal is to address these information needs in a retrieval system that returns results efficiently and effectively. However, we believe the methodology, can be applied to other information needs within the medical domain, forming a generalizable framework.
We first identify common axes along which medical research is typically conducted for a specific intent, such as temporal aspects or project phases. Along these axes, our prompting framework enables a targeted approach to generate synthetic test datasets of natural language questions paired with SPARQL queries. Specifically, clinical trials typically undergo different phases, such as: *Trial Eligibility*, *Study Design*, *Intervention Details*, *Trial Outcomes*, *Location and Logistics*, *Trial Sponsors and Investigators* and *Results and Publication*. Another important axis are standardized roles within a clinical trial to ensures independent oversight and that all medical aspects of the research are conducted by qualified personnel, as required by standards such as the *Good clinical practice* (GCP) international standard. The standard emphasizes the clear separation of tasks and roles as a non-negotiable principles [8], section 10.1.

A primary investigator for example conducts the clinical trial and must ensure that its execution is performed in a practical, scientific, regulatory and ethical manner. When conducting the trial (*Intervention* and *Outcome Analysis*), the person responsible for conducting the clinical part of the trial must monitor events and record and report the outcomes. In terms of information needs, this typically results in questions about patients with a specific diagnosis within certain cohort, checking whether patients have complied with intervention procedures, such as taking prescribed drugs or substances at scheduled intervals, the percentages of patients with labresults above a certain threshold and identifying patients who dropped out of the trial.
Other roles typically involved in clinical trials are the *Investigator*, *Research Nurse*, *Clinical Data Manager*, *Clinical Research Nurse* and *Pharmacist* [9]. In the design of information and AI systems, research shows that modeling user intent substiantially improves precision and recall when designing classes of information needs for knowledge driven retrieval and classification tasks [10]. Based on these insights, we propose a comprehensive framework, that we believe is expressive enough to cover a siginificant part of medical information needs necessary during a clinical trial. This, in turn, facilitates the design of a natural language question (NLQ) to SPARQL pipeline, that can adapt to user intent for improved performance of generating SPARQL queries and query performance. Since the framework derives it's structure from regulatory documents and standard guidelines, it provides substantial coverage of the information needs of clinicians and related specialists involved in conducting clinical trials.
In the methodology section, we illustrate the prompting steps we took to generate the synthetic dataset, based on the evolution of the project. The starting point was a sample of an anonymized dataset, based on a simplified version of the SPHN ontology. We believe that the steps taken are promising and can be converted into a generic method applicable to other use cases as well. In our case, the values that comprises the axes closely aligned with medical standards.

# 2 Related Work

This section tackles the four general sections of the project that are the effect of user intent in graph databases, natural language question generation, logical query generation and the graph engine to be used on those logical queries. In general for each, we look at alternative work that has been done in that sector and how we can apply or use it for this project. If none of these methods are chosen then, the reasons why are given along with what approach most closely suits the needs of the project which would inspire the eventual methodology.

## 2.1 User Intent in Graph Databases

An important subject in this research is the identification of user intent in the context of graph database querying. Although extensively studied within the domain of search engines, this topic remains relatively underexplored in question answering systems, which constitute the primary focus of this study. The data used for the creating user intent is usually very vast collecting macro data of general user behaviour and more micro data on the click patterns of a user. While this is not entirely applicable to our system, the general idea of improving user experience is the same. There are numerous cases where a marked improvement is seen through the usage of user intents in creating easier pathways through a vast dataset. Ren, et al. [11] show a graph based clustering method used to increase performance of search tasks. In this project there is a similar intent behind the usage of User Profiles where the aim is to search through the most likely question patterns to quickly understand the intent of the question posed by the user.

## 2.2 Natural Language Question Generation

This phase involves creating a testing set of questions that are answerable based on the data used for the creation of the PHKG. Additionally, the questions should be separated by user type as is defined later. The queries must also be of varying complexities to ensure that an exhaustive coverage of the dataset is considered. This is necessary to make sure that the system is not found lacking when presented with a more complex question than would be typically expected. Primarily for this case, union and compound conditions added to multiple existing query question pairs.[12].

### 2.2.1 MIMIC EHR QA

This method involves using the MIMIC III database of electronic health records as a basis for the formation of our natural language question as outlined by Wang et al. [13] and as used by Park et al.[14]. MIMIC III stores the records in a relational database of 26 tables that contain various information on a patient. Namely, their hospital level data, ICU level data, emergency department data, metadata and apart from these there are also some de-identified clinical notes.

These tables also contain information on the diagnosis, procedures, dosage, and prescription as well as events preceding the appointment and some lab results. These tables are then reduced and simplified to contain 5 points of data, namely Demographics, Diagnoses, Procedures, Prescriptions, and Labs. However since there are duplicates within this data as the aforementioned department databases are related but not dependent on each other. Park et al. used a method by Pampari et al.[15] that generates NLQ-SQL pairs which would be the template questions to be used by randomly sampling. There are some limitations however as these questions do not completely replicate the linguistic diversity of a human question which also includes generally a less specific query which is what a human is most likely to ask. Although Park et al.[14] did not account for the variable complexity of a human question, it is possible to account for this by noting the number of conditions in the SQL query which would allow for a decent representative sample of different levels of complexity of a question to be present within the dataset.

While, this approach on the surface seems very effective for the purposes of this project, its limitations lie in the lack of curation of question templates which would not allow for the effective creation of a definitive user profile. Additionally, there is also a lack of an accuracy check that ensures the questions generated from the template are answerable on the graph. For these reasons and the fact that the questions themselves were based off a different, larger and less focused subset of the sphn dataset, this set is not preferred.

### 2.2.2 Cypherbench

This method as outlined in the paper by Feng et al. [16] creates question, Cypher pairs and converts them to a more natural form using an LLM. These Cypher queries are generated through the traversal of the graph by some preset templates. In the aforementioned paper, seven basic graph patterns are considered as they cover all possible isomorphism structures for a typical EHR and lead to at least one answer which would be useful when testing. Five special graph patterns are also added to this list that cover comparison, grouping, optional matching, time based queries and union and these cover the various methods for adding complexity to a human question.

The major limitation to this approach, similar to the previous method is that it does not account for more general questions, however unlike the previous method, this is not because of the lack of linguistic diversity but the potential dependency on a large number of documents that would not be covered in the graph templates. However, the implication of this limitation is reduced somewhat by the general

assumption that the queries being asked to the model will be related primarily to medicine and as such any questions straying from that topic would not be an especially pressing concern and not worth testing. This same reasoning applies to the user profile case where it is even more relevant as questions that are unlikely to be asked by the profile will only hinder the system. This is why this dataset, although fast and effective, is not suitable for the purposes of this project.

## 2.3 Logical Query Generation

Translating the aforementioned natural language questions into executable logical forms—such as SPARQL [17] and First-Order Logical queries—is a crucial step in Knowledge Graph Question Answering (KGQA). This process involves converting the user's input questions into formal expressions (i.e., queries) that a knowledge graph processing engine can interpret and execute.

However, generating these logical queries poses significant challenges. First, as knowledge graphs grow in size and complexity, crafting accurate SPARQL queries becomes increasingly difficult. Recent studies indicate that large language models (LLMs) often struggle with reasoning over extensive inputs [18, 19, 20, 21]. Moreover, complex questions may involve nodes that are distantly related, thereby requiring multi-hop reasoning—a well-known challenge for KGQA systems [12]. Additionally, real-world knowledge graphs frequently suffer from incompleteness, which can make it impossible to answer complex questions by merely traversing the graph when essential links are missing [22].

To address these challenges for this project, several innovative frameworks have been considered:

- **SPARQL-LLM** [23]: This framework leverages metadata from knowledge graphs—including example questions and schema information—by first retrieving and indexing minimal standardized metadata from a set of SPARQL endpoints. It then employs an embeddings-based similarity search to extract the most relevant context, which is used to construct a prompt for the LLM. The initially generated query is subsequently validated and refined by cross-referencing it with the endpoint's schema, ensuring both accuracy and consistency before presenting the corrected query to the user.
- **CDQ (Continuous Query Decomposition)** [24]: CDQ efficiently answers complex questions over incomplete knowledge graphs by translating each question into an end-to-end differentiable objective. Each atom's truth value is computed using a pre-trained neural link predictor, and the scores are aggregated via continuous relaxations of logical operators. This approach effectively formulates the output as a First-Order Logical query, mitigating the challenges posed by missing links.
- **QTO (Query Computation Tree Optimization)** [22]: QTO optimally addresses complex logical questions by converting a natural language question into a computation tree—a structured representation that encodes the question's structure and dependencies. A forward-backward propagation strategy then exploits the tree's local independence, significantly reducing the search space and enabling efficient, interpretable query answering. Importantly, this computation tree is semantically equivalent to a First-Order Logical query, preserving all logical relationships.

For the purposes of this project, the approach deemed to be most well suited is the SPARQL-LLM method. This is because the primary concern for this project is the formation of accurate SPARQL queries in relation to the user profile, the performances of the queries or efficiency of the query is a secondary concern. This means that the ignoring of missing links in CDQ poses a major issue as we want to ensure there are no false results and we know what most questions asked by the user type would be as they are based off the patterns defined prior, which ensures that a SPARQL-LLM inspired method is best suited for the project's purposes.

## 2.4 Graph Query Engine

Our project already includes a dataset mapped to SPHN, structured as RDF triples. To ensure efficient query processing and database management, we need a robust open-source engine capable of handling large-scale graph data. Our pipeline involves generating logical forms from natural language and executing them on our personal health knowledge graph. As our knowledge graph grows—potentially reaching many triples—our database engine must efficiently process and query the data. Given that our data is represented as an RDF graph and our queries are based on logical forms, we need an open-source query engine and robust analytical tools to evaluate both query effectiveness, query shapes and engine performance.

### 2.4.1 AvantGraph

AvantGraph is a high-performance query engine optimized for both subgraph matching and navigational queries [25]. It introduces Magellan, a top-down query planner, BallPark for cardinality estimation [26], and QuickSilver, an execution engine inspired by a recent QuickStep scale-up data platform [27]. Internally, AvantGraph translates queries into regular property graph algebra (RPGA) [28], allowing it to handle both SPARQL and Cypher queries efficiently. It supports both RDF and Property Graph data models and integrates advanced graph algorithms such as worst-case optimal joins [29]. Additionally, its Tune and Explain toolkit provides valuable insights into query execution strategies, enabling fine-tuned performance analysis and optimization.

### 2.4.2 MillenniumDB

MillenniumDB is an open-source graph database system designed for efficient query processing based on a graph data model called domain graphs. Using this abstraction, it supports a variety of graph models, including both RDF and property graphs [30]. Like AvantGraph, MillenniumDB implements worst-case optimal joins to enhance performance when processing complex queries. Its architecture is designed to scale with large datasets while maintaining efficient retrieval and update operations. MilleniumDB's server architecture is robust, handling both

### 2.4.3 QLever

QLever extends SPARQL with built-in text search capabilities [31]. Developed at the University of Freiburg, it enhances standard SPARQL queries with operators such as `ql:contains-entity` and `ql:contains-word`. This integration allows QLever to perform well in both pure SPARQL queries and hybrid SPARQL + text search queries. When compared to popular engines like Virtuoso, QLever demonstrates superior performance in scenarios requiring combined structured and unstructured data retrieval.

Among these query engines, MilleniumDB stands out. It is fully open source, unlike AvantGraph which is currently still in the process of becoming open source. Furthermore, its strong performance in benchmarks make it an attractive choice.

# 3 Methodology

This section provides a detailed overview of the different steps of the project, the research questions they address, the hypotheses they aim to improve, the corresponding solution(s) and the approach how to measure the performance of the solution(s).

## 3.1 Representative question-query templates

To generate a representative set of SPARQL query templates for different user intents for certain profile, a curated set of question-query template pairs is required. Importantly, the format of the SPARQL templates must be suitable to obtain information from the specific knowledge graph, meaning that the query must align with the ontology presenting the graphs structure. This section introduces a prompt-and-curate approach towards the generation of a question-query set.

### 3.1.1 Synthetic Natural Language questions-SPARQL Query pairs

Since no user logging is available for this project, other ways need to be found for us to be able to work with the test data. This results in the research question:

**1** ***Given an existing ontology, can we find a method to generate a relevant set of natural language question-SPARQL query pairs for question answering on a Personal Health Knowledge Graph for a specific user intent?***

To solve this, we believe that it is possible to generate representative natural language question-SPARQL pairs based on a structured approach that is generalizable.

**User Profiles and Intent** The first prompting step was aimed at generating a general starting point of a question set for clinical trials, independent of the given knowledge graph, to understand the general coverage afterwards. We started with the general prompt using ChatGPT 4.o:

```
Please provide: example SPARQL or SQL queries a
    trial conductor might issue in a data-driven platform like SPHN.
```

We verified with a follow-up prompt:

```
A clinical person that conducts a clinical trial,
    are its information needs covered by these questions,
    or do they need to be expanded?
    {...list of the four generated questions from the previous step...}
```

The next question was targeted at zooming in into the specific set we wanted to work with, with the prompt

```
Please generate a synthetic dataset of Q\&A pairs based on SPHN RDF data.
```

Since we knew how fundamental the role is in a clinical trial is, we shifted towards fixing one of the axes related to roles in a clinical trial:

```
Explain all roles involved in actively conducting a clinical trial
    that might have medical information needs
```

Similarly, we prompted for phases in a clinical trial and intent per role. Finally, we asked for identifying each axes values in a central overview:

```
Is it possible to provide an over view of all relevant intent categories
    in the context of clinical trials, by trial phase and user role?
    If so, please provide the overview.
```

The resulting dataset was our foundation for generating query templates, resulting in a 3-axis overview A.1. With that framework of targeting user profiles and intent, the following prompt generated a list of SPARQL queries relevant for that specific context and intent

```
Provide a list of query patterns for:
    {...list of all axes values...}
```

The SPARQL queries returned were manually validated in terms of compliance with the ontology and for syntactical correctness. The full list of the 8 template patterns generated, curated and used can be accessed in A.2.

## 3.2 SPHN Knowledge Graph

This project is centered around a knowledge graph extracted from a subset of the MIMIC-IV database using the sphn schema. This dataset was provided to us as a motivation to perform analysis on efficient query generation. The original knowledge graph was complete and well connected but lacked predicates, classes and values necessary to cover information needs targeted at a specific user profile and intents. Additionally, some of the concepts provided represented nested concepts. This turned out to be an issue, since central classes such as patients are central concepts to be returned as isolated instances as query results. As such, there was a need to add more information to the graph. Furthermore, Internationalized Resource Identifiers (IRIs) of some concepts were not provided as standalone identifiers but were instead embedded within relational structures and nested concepts. For example, a drug entity could only be identified through its association with a specific patient to whom it was administered, rather than through a unique, independent identifier. In this representation, entities are defined and discoverable solely via their relational context. This greatly limits the number and variety of questions that can be asked and as such, each instance of these types of classes were given their own unique identifier. Therefore, starting from the provided knowledge graph, missing classes and predicates were identified and added to cover a minimal ontology that contains answers to a minimal set of question-query pairs for the user intent, user profile and phase of the trial that was most covered by the original knowledge graph as part of a custom domain, that was introduced to fill gaps where the original SPHN knowledge graph was incomplete and no corresponding concept or predicate was found after a brief search within the official SPHN ontology.

Secondly, for classes and predicates that did not have actual instances in the given class, random values were generated. Equally, for the newly added classes and predicates values were added. For example a randomized birthdate was given in the range of 5 to 90, given the year 2024 and patients were given a randomized set of diagnoses.

## 3.3 Pipeline

In this section, we introduce the Knowledge Graph Question Answering (KGQA) pipeline and its individual components. A KGQA system that receives natural language questions, and creates and executes logical forms to retrieve relevant context is not a new idea and has been explored across various domains, often through the use of natural language interfaces to structured data [32][33][14]. In general a process like this consists of three steps: posing a natural language question, generating a relevant logical form, and retrieving the corresponding triples from the knowledge graph, as shown in Figure 1.

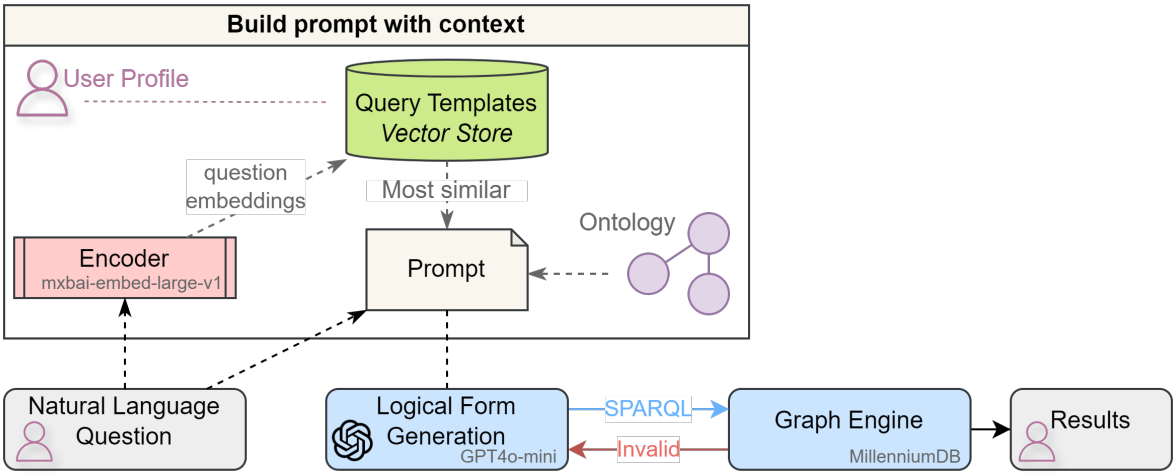


Figure 1: Diagram illustrating the end-to-end pipeline for knowledge graph question answering (KGQA), from input processing to SPARQL query execution. The diagram emphasizes the contextualized prompt construction, which is the core distinguishing feature of our approach.

### 3.3.1 Logical Form Generation

To obtain an answer based on factual information from the knowledge graph, a natural language question should be transformed into a logical form directly executable on the graph engine.

**2** ***Given a natural language question, how can we create optimal logical forms that accurately retrieve relevant context from a specified Personal Health Knowledge Graph.***

The generation of an logical form is commonly done by utilizing large language models (LLMs). While LLMs offer powerful capabilities, their performance is highly dependent on prompt design and the quality of contextual information provided. To address this, our prompting strategy incorporates multiple refinement steps, as illustrated in Figure 1.

**Baseline** To evaluate the effectiveness of our approach, we define a baseline using a minimalistic prompt with limited guidance. The baseline prompt, shown in B.2, includes three components: (1) a predefined example SPARQL query, (2) the relevant ontology B.1, and (3) the input natural language question. This approach relies entirely on the intrinsic reasoning ability of a decoder-only model (GPT-4o Mini), without any retrieval augmentation, fine-tuning, or post-processing. The output of this baseline is a raw SPARQL query generated directly from the model.

**Contextualized Prompt** Inspired by the SPARQL-LLM approach [23], we enrich the prompt with context specifically tailored to the input question. This method is particularly effective in our setting, where user profiles enable the provision of a limited but representative set of user questions. As a result, we can include well-matched examples, paired with their corresponding templates, thereby reducing

the reliance on the LLM to generate novel query logic and minimizing the risk of hallucinations. The contextualized prompt includes: (1) the ontology, (2) question-relevant query templates, and (3) question-relevant examples.

To obtain question-relevant context, we utilize a vector store that contains a set of representative question–query pairs, containing for each template a concrete examples. This vector store makes use of the open-source encoder model mxbai-embed-large-v1.

For the query templates vector store, the system retrieves the top 3 most relevant question–query pairs by encoding the input natural language question and comparing it to the embeddings of template questions. These pairs contribute both template-based and example-based SPARQL queries to the prompt.

The contextualized prompt is shown in B.3

In addition to evaluating the contextualized prompt, we also analyze performance using a baseline prompt that includes only a single question–query example relevant to the input, but omits any template information. This setup allows for a clearer comparison between our approach and SPARQL-LLM [23], highlighting the performance gains achieved by incorporating structured templates and richer contextual information.

**Evaluation** The performance of logical form generation is evaluated using a curated test set of 210 question–answer (Q&A) pairs. These test questions are automatically generated by prompting the LLM, as described in B.5. Each test question and filled in query template is based on the representative question–query template from the user profile A.2. The test questions are reformulations of the original question templates, enabling a realistic evaluation of the model's ability to generalize across the covered set of representative questions to naturally phrased user inputs.

The test set consists of 15 questions each for templates 2 and 5, and 45 questions each for templates 0, 1, 3, and 4. *Because the lack of data; question–query template 6 and 7 are not present in the test set.*

Evaluation of the queries is based on two key criteria: (1) the execution speed of the generated query on the graph engine, and (2) the correctness of the query, determined by whether the returned results match the expected answers. We analyse over four evaluation metrics: **Precision**, **Recall**, **Accuracy**, and **F1-Score**.

- **Precision** is defined as the number of correctly retrieved instances from the PHKG that match the expected results, divided by the total number of retrieved instances.
- **Recall** is defined as the number of correctly retrieved instances divided by the total number of expected instances.
- **Accuracy** is the proportion of test questions for which the generated query returns results that exactly match the expected answer set.
- **F1-Score** is the harmonic mean of Precision and Recall, providing a single measure that balances both aspects.

In addition to evaluating query optimality, we also measure the end-to-end duration of the query generation process. This includes the time required for (1) constructing the prompt (contextualized or otherwise), (2) executing the prompt using the LLM, and (3) performing any necessary post-generation validation.

In addition to evaluating the overall logical form generation pipeline, we assess the performance of the underlying vector store and embedding model, focusing on how effectively relevant query templates are retrieved.

For template question–query pair retrieval, we evaluate for each test question how accurately the vector store surfaces relevant examples. Specifically, we report the ranking position at which the actual relevant template is retrieved. Ideally, the correct template should appear at the top (rank 1) for each test question. This metric quantifies how well the vector store prioritizes semantically similar question–template pairs, taking in count more examples could be provided to the prompt.

### 3.3.2 Graph Engine

This module is responsible for executing the SPARQL queries generated by the system on the PHKG. Its primary function is to ensure that the system can reliably interact with the knowledge graph and quickly return results in response to user queries. Efficient execution of queries on the graph data will enable scalable performance as the PHKG expands in size and complexity.

**3 *How can the logical forms be efficiently executed on the Personal Health Knowledge Graph?***

To ensure efficient execution of logical forms, two key conditions must be met: (1) the logical form must be expressed in a format compatible with the graph engine, and (2) the graph engine must be capable of executing the query effectively. While the generation of logical forms lies outside the graph engine's scope, the forms must still be validated and, if necessary, corrected to be executable. This necessitates the introduction of both a validation layer and an execution layer within the system architecture.

The execution layer employs a dedicated graph engine to support high-performance query processing. Positioned between the logical form generation and execution layers, the validation component plays a critical role in ensuring that only executable queries are forwarded for evaluation. For this purpose, MillenniumDB was chosen as the graph database engine due to its robust performance and built-in capabilities for detailed query diagnostics. These features are particularly valuable in systems that must efficiently handle queries generated by large language models, which may occasionally be syntactically or semantically malformed.

For the execution layer, a dedicated graph engine was integrated into the system, with the goal of supporting fast query execution. In between the execution layer and the logical form generation, a validation step was incorporated. MillenniumDB was selected as the graph database engine for this layer due to its efficient query execution performance and built-in support for query error diagnostics. MillenniumDB has demonstrated faster query execution times compared to popular open-source graph engines such as Neo4j and BlazeGraph [30].

MillenniumDB fulfills two central functions within the system. First, it acts as the primary interface to the PHKG, executing valid SPARQL queries and returning results. This ensures efficient execution of the queries. Second, in cases where a query fails to meet correct SPARQL syntax, MillenniumDB provides detailed diagnostic feedback, rather than a binary success/failure response.

These diagnostic messages are then incorporated into a validation prompt B.4 and routed back to the validation and correction stage, where they can inform subsequent query refinement attempts. This happens up to three times. The validation and correction stage aims to help to maintain execution integrity while enabling iterative improvement.

Research Question 3 is addressed by evaluating both the performance characteristics and the diagnostic feedback capabilities of the execution engine in the context of SPARQL queries generated by the system. The evaluation focuses on execution latency, validation overhead, and the utility of error diagnostics provided by MillenniumDB. Through this analysis, the study aims to determine whether combining query validation with execution can yield a robust and efficient mechanism for handling logical forms within the PHKG framework.

# 4 Results

This section will show and discuss briefly the results of the tests outlined in 3.

## 4.1 Representative question-query templates

In order to assess the coverage of the generated natural language question-SPARQL query pairs A.2, we turn to scientific literature about trials to get a qualitative estimate of the success of this method across the previously discussed axes user role, trial phase and user intent as an exemplary analysis. For the user intent of eligibility, as described in [34], patient characteristics such as gender or pregnancy status and co-morbidities are missing together with a list of other characteristics. However, these patient characteristics were not provided in the original dataset and therefore not applicable. Additionally, queries that require this information are typically in the form of AND/OR accumulations of medically relevant characteristics across patients. Similarly, good practice suggests similar characteristics. From what we saw, most significant is the missing patterns of temporal context, such as a patient's last doctors visit and diagnoses are not older than a certain number of month, for example not older than 6 month [35].

## 4.2 Logical Form Generation

This section provides insights into the performance improvement of the Logical Form Generation having an contextualized prompt.

### 4.2.1 Query Template Selection

Before evaluating the performance of logical form generation using a contextualized prompt, it is important to first assess the quality of the context provided to the LLM.

The vector store successfully retrieves the correct (target) template as the top-ranked result for nearly all test questions. The only exceptions are two cases under Template 0, where the correct template was ranked second and third, respectively. This confirms that the contextualized prompt—which includes the top three most similar templates—consistently contains the correct template.

This near-optimal retrieval performance supports the choice to include the top 3 templates in the prompt rather than relying solely on the top 1, as it demonstrates that retrieval errors can still occur—even within a small representative set. Moreover, it strengthens confidence in the scalability of the approach when applied to a larger and more diverse template collection.

### 4.2.2 Query Generation

In order to measure the effectiveness of the system at generating queries as generally illustrated by the entry in appendix B.7, its performance is measured across 4 different criteria.

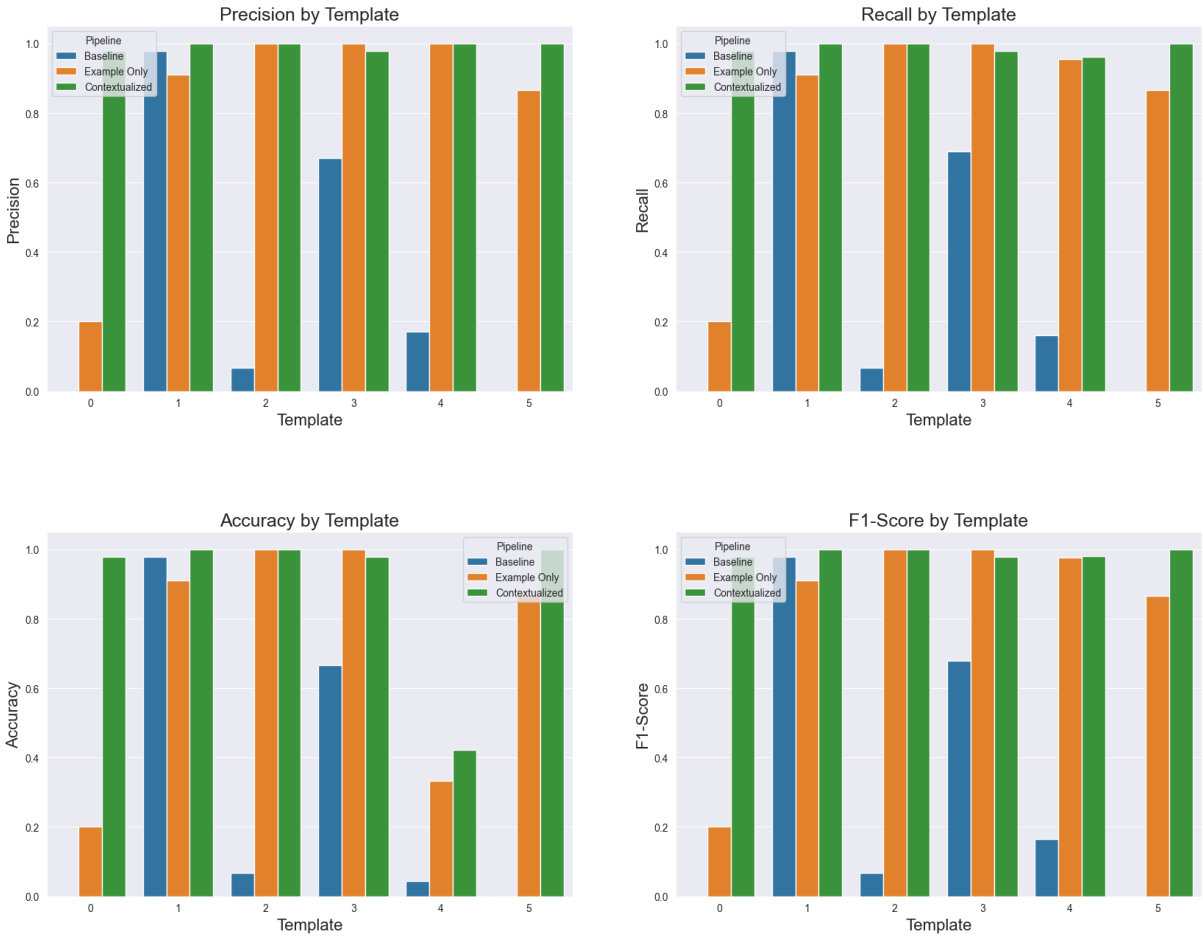


Figure 2: Evaluation metrics by template for each pipeline

**Query Correctness** Figure 2 presents a comparative analysis of pipeline performance across six SPARQL query templates. The pipelines are evaluated on four key metrics: Precision, Recall, Accuracy, and F1-Score. For each template, the average performance per, from the templates, generated questions is shown side by side across the three pipelines.

In terms of **Precision**, the *Contextualized* pipeline consistently outperforms the others, achieving near-perfect results, with only a slight dip compared to the *Example Only* pipeline on Template 3. The *Example Only* pipeline performs almost on par with the *Contextualized* one overall but struggles

significantly with Template 1. The *Baseline* pipeline demonstrates relatively poor performance across most templates, with the exception of Templates 1 and 3, where it shows somewhat better results.

A similar trend is observed for both **Recall** and **F1-Score**. In fact, the F1-Score curves closely mirror those of Precision and Recall, indicating balanced contributions from both metrics. However, **Accuracy** diverges slightly, particularly for Template 4, where both the *Example Only* and *Contextualized* pipelines show noticeably lower Accuracy than what their Precision and Recall scores might suggest.

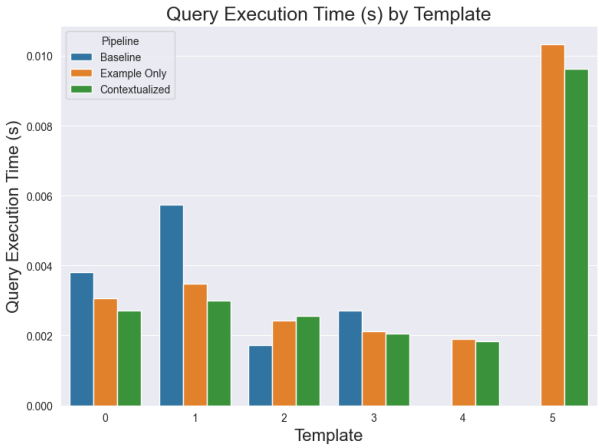


Figure 3: Query execution time by template for each pipeline

**Query Execution Speed** The Figure 3 shows the average time taken to execute the final query generated by each pipeline on the PHKG in MillenniumDB, grouped by the template used to generate the natural language questions. To ensure a fair comparison, only queries that produced an exact match with the expected output were considered.

Overall, query execution was very fast, with most queries completing in under 0.01 seconds. The *Baseline* pipeline exhibited the fastest execution times but did not produce any valid results for Templates 4 and 5. The other two pipelines demonstrated similar performance, with the *Contextualized* pipeline being slightly faster. Notably, queries corresponding to Template 5 took significantly longer to execute than those for the other templates.

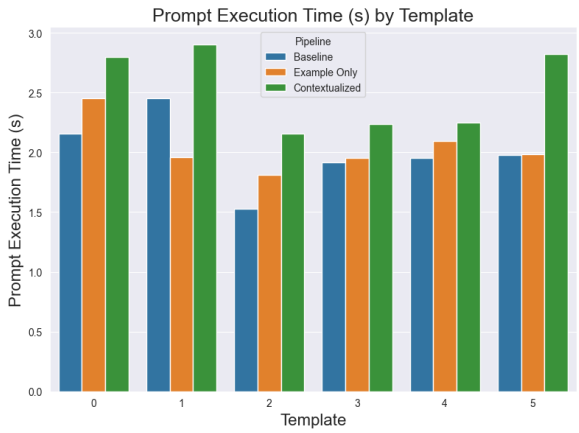


Figure 4: Average prompt execution time per approach, broken down by question–template group.

**Query Generation Speed** As shown in Figure 4, the prompt execution time increases with the amount of context included in the prompt, which is expected behavior for an LLM. The baseline prompt, containing minimal context and a generic SPARQL example, achieves the shortest execution time. The example-only prompt, which replaces this example with a single question-specific example, performs slightly slower than the baseline due to the added length and specificity of the example. Both of these

approaches significantly outperform the fully contextualized prompt, which includes three examples and three templates, resulting in a considerably longer prompt and thus higher execution time.

In addition to the prompt execution time, both the contextualized prompt and the example-only prompt require an extra step of encoding the input question and retrieving the most similar example and/or template from the vector store. While this step introduces additional overhead, it remains relatively lightweight: on our local machine, the retrieval process averaged only 0.2 seconds per query. This latency could be further reduced when deployed via a dedicated retrieval endpoint or optimized infrastructure.

**Validation Process** To evaluate the impact of incorporating query-specific error messages into the prompt, two validation strategies were compared on a curated set of questions containing various SPARQL syntax errors. Both execution time and the ability to generate syntactically valid SPARQL queries were assessed under two distinct conditions:

1. **Generic Error Message**: A general error message was provided to the model, indicating a syntax issue, as follows: *"SPARQL Syntax Error: The format of the query is incorrect."* This message was included in the validation prompt detailed in Appendix [REF].

2. **Specific Error Message**: A more detailed, query-specific error was supplied, obtained through MillenniumDB's SPARQL syntax verification tool. An example of such an error is: *"Bad query semantic: 'The prefix 'sphn' is not defined.'"*

The inclusion of specific error messages significantly improved the model's ability to correct queries, often requiring fewer validation attempts. In cases where both approaches required the same number of attempts, the specific-message condition resulted in only a marginal increase in response time—typically under one second. Notably, certain errors that could not be resolved within three attempts using the generic message were successfully corrected when the specific error was supplied (see Table 1).

| | **Generic** | | | **Specific** | | |
|---|---|---|---|---|---|---|
| **error type** | **Time** | **Valid** | **Attempts** | **Time** | **Valid** | **Attempts** |
| extra dot | 1.608273 | True | 1 | 1.715380 | True | 1 |
| missing prefix | 6.667113 | False | 3 | 2.040747 | True | 1 |
| SEL instead of SELECT | 1.597018 | True | 1 | 1.384734 | True | 1 |
| Incorrect prefix keyword: spn instead of sphn | 1.808605 | True | 1 | 1.370281 | True | 1 |
| Missing '{' | 3.328844 | True | 1 | 2.345854 | True | 1 |
| Used icd: instead of string | 8.614784 | False | 3 | 6.378679 | True | 3 |

Table 1: Comparison of query correction performance using generic versus specific error messages. Specific error strings consistently lead higher success in generating valid SPARQL queries and fewer validation attempts

An aggregate comparison across the full test set further demonstrated that the use of specific error messages not only improved correction success rates but also yielded better average correction times (Figure 5).

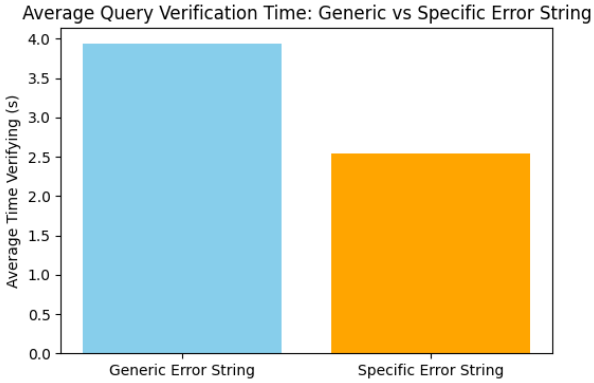


Figure 5: Average query verification time across the full test set for the two prompt types. The use of specific error messages results in substantially lower average verification time compared to generic error messages, indicating improved efficiency in resolving SPARQL syntax issues.

# 5 Discussion

In this section we will discuss results and their relevance to our research questions.

## 5.1 Representative question-query template

Based on our research, most of the logical patterns of asking questions could be found with our prompting method, such as logical AND/OR questions for a combination of diagnoses of patients. The actual values of characteristics are constraint by the availability of data by design and not a flaw of the method. The expected values of the patient characteristics are considered got close to be covered by our method. The main missing gap we found is about some more complex questions that connect temporal information with patient characteristics, such as a patient with a specific age range and comorbidity had a doctors visit in the last 6 month, to be able to include or exclude a potential trial participant during the selection phase.

## 5.2 Prompt Performance

Overall, the results support the hypothesis presented; the *Contextualized* prompt strategy leads to improved performance across nearly all metrics. It effectively balances relevance and precision through richer question based context.

An intriguing observation arises in the case of Template 4 A.2: both the contextualized pipeline and the example-based pipeline demonstrate high precision and recall when the results of their generated queries are evaluated against the results of the target query. However, despite obtaining many of the same results, the generated queries rarely constitute exact matches with the expected template instance. In most of these near-matches, the generated queries return strict subsets of the results retrieved by the target query. This leads to perfect precision and near-perfect recall.

A closer inspection of the near-match queries reveals a consistent pattern explaining this discrepancy. Specifically, the placeholder in the template that the language model must substitute with an appropriate value is embedded in the following expression:

```
SELECT DISTINCT ?patient WHERE...
...
...FILTER(?age >= §Age§)
```

In the non-exact matches, this expression is always transformed into:

```
SELECT DISTINCT ?patient WHERE...
...
...FILTER(?age > §Age§)
```

Importantly, the language model correctly identifies and substitutes the appropriate value for the `§Age§` placeholder. However, it also modifies the comparison operator, replacing `>=` with `>`, which is

outside the bounds of the marked placeholder. This change has meaningful semantic consequences and the resulting query excludes entities that exactly match the specified age threshold, thereby reducing recall.

This phenomenon highlights a limitation in the current prompt design: while the model correctly interprets and substitutes values within designated placeholders, in this case, it sometimes modifies content outside these delimiters, inadvertently altering the query logic. Such behaviour suggests that the model does not treat the non-placeholder portions of the query as immutable, leading to unintended deviations from the intended semantics.

The implications of this finding are twofold. First, it underscores the importance of carefully designing prompt instructions to enforce stricter adherence to template structure. Second, it suggest that the correction and verification step might be extended to verify that queries belong to an existing query template.

**Baseline** In contrast, the *Baseline* pipeline consistently underperforms, except in cases where the test questions closely align with the fixed example it relies on. This highlights the limitations of using only the ontology and a single example for reasoning and generalization. The results point to a key shortcoming in the current ontology and its associated graph structure, both of which were developed quickly as a temporary solution. The ontology lacks sufficient structure and semantic richness to clearly convey the relationships and entities in the graph. As a result, it fails to provide the level of clarity and guidance needed for effective reasoning by the LLM.

## 5.3 Limited Representative Set

The current representative question–query set, developed for the selected user profile, remains relatively small. While this allows for focused evaluation, it also imposes two important limitations. First, only a narrow range of question types can be accurately answered using a matching template from the set. Second, the small size of the representative set increases the likelihood that the vector store retrieves the correct template for a given test question—especially since these test questions are derived from the representative questions themselves.

As a result, the performance metrics of the contextualized prompting approach observed in our evaluation are likely an overconfident. In a more realistic deployment scenario with a larger and more diverse representative set, retrieval accuracy may decline and the overall system would depend more heavily on the LLM's reasoning abilities to fill in gaps.

## 5.4 User Profile Separation

Although there is only one user profile used in this project there is the possibility of having more user profiles made that effectively utilise the vector store which would allow the vector store to be small and effective. This delineation of the vector store across user profiles would make this system very scalable as irrespective of the size of the dataset, the performance of the system for any given user should be very similar across dataset sizes.

## 5.5 Query Generation Speed

A key objective of our system is to optimize the overall speed of query generation. While fast SPARQL query execution is important—especially as the graph size increases—our observations show that the actual execution time of the SPARQL query is relatively low compared to the time consumed by upstream tasks. These include contextualizing the prompt, executing the prompt via the LLM, and validating the generated query.

Currently, the contextualized prompt introduces a longer execution time due to the inclusion of various supporting elements: 3 relevant templates and examples, and the ontology. To reduce overall prompt execution time, likely resuling in faster prompt execution speed then the baseline, we propose simplifying the prompt content. For such simplified template based prompt we could reason it would still perform quite well by (1) removing the ontology and (2) including only one template and example pair

Excluding the ontology is still likely to be effective, as evidenced by the poor performance of the baseline approach—which relies heavily on the ontology—and the near-optimal results achieved by the contextualized prompt that emphasizes the use of templates. Given this strong performance, along with the limitations of the current, poorly structured ontology (as discussed in subsection 5.2), it is reasonable

to conclude that removing the ontology would have minimal impact on overall performance. At the same time, it would contribute to a faster and more efficient prompt execution process.

Only incorporating one example is also unlikely to negatively impact performance for the vast majority of questions. This is supported by our analysis showing that the correct template was ranked as the most similar (top-1) for 208 out of the 210 test questions. This strong performance suggests that, even in a reduced form, the contextualized prompt retains enough relevant context to guide the LLM effectively while significantly improving execution speed.

### 5.6 Query validation

The use of specified error prompts substantially enhanced the correction layer's ability to generate valid and executable SPARQL queries. Incorporating query validation positively impacted query success: all initially invalid queries became executable within three rounds of validation when using the MillenniumDB error prompt. While these results are promising, it is important to note that the validation step was not utilized in the contextualized prompt strategy and was applied only once in the example-based prompt strategy. This suggests that prompt strategies relying on query templates enable large language models to produce syntactically valid SPARQL with high reliability, particularly when the user's question closely aligns in intent with an existing template or example

## 6 Conclusion

This project explored the development of a representative set of question–query template pairs tailored to specific user profiles and their application in generating optimal SPARQL queries that can be directly executed on a knowledge graph. By leveraging this representative set—composed of templates and example pairs—the system was able to guide large language models (LLMs) in producing accurate and often near-optimal logical forms for diverse user questions.

The use of contextualized prompts based on this curated template set led to a consistently outperforming pipeline, demonstrating the effectiveness of grounding query generation in structured, user profile-specific examples. While these are promising initial results, incorporating multiple user profiles and larger ontologies remain open challenges where the techniques used in this paper are not trivial to extend to.

The corresponding research questions addressed in this work are:

1. **Given an existing ontology, can we find a method to generate a relevant set of natural language question-SPARQL query pairs for question answering on a Personal Health Knowledge Graph for a specific user intent?**

   Using prompting methods, we successfully generated relevant natural language question-SPARQL query pairs. A qualitative evaluation based on medical literature showed gaps between the concepts, such as a users specific comorbidities and the information need for a specific user intent. However, these gaps can be explained by the limitation of the dataset not by a flaw of the method. An actual gap was identified in terms of temporal aspects in combination with concept values of class instances, such as if a patient visited a doctor in accordance with a predefined schedule, or if a patient with a specific diagnosis visited the doctor within the last 6 month.

2. **Given a natural language question, how can we create optimal logical forms that accurately retrieve relevant context from a specified Personal Health Knowledge Graph?**

   In this research, we extend prior work on logical form generation by introducing a novel strategy for the collection and representation of example sets. Our contribution is twofold: first, we propose the use of a curated set of question–query template pairs, specifically tailored to a user profile and its associated query intents. This enables the system to focus on generating efficient query structures that directly align with the user's characteristic information needs. Second, by narrowing the scope to a single user profile, it becomes feasible to exhaustively represent its query intents through templates. This allows for the construction of highly curated and efficient query forms, enhancing performance compared to generating queries from scratch. By leveraging templates rather than relying solely on example-based learning, the query generation task is reframed as the instantiation of a predefined template in response to a natural language question. Empirical results indicate that this template-driven approach yields more consistent success in generating the intended efficient queries, outperforming example-based methods such as SPARQL-LLM.

3. **How can the logical forms be efficiently executed on the Personal Health Knowledge Graph?**

   To enable efficient execution of logical forms on the PHKG, we introduce an execution-validation layer. The layer validates logical forms, returning an error for incorrect queries and a prompt to correct it. If a query is executable, the layer will retrieve the results from the PHKG. We leveraged MilleniumDB as the underlying graph engine due to its fast query execution and error diagnostics. Empirical results show that incorporating specific error messages into the validation prompt significantly improves correction success rate, reduces the number of validation attempts and the overall validation time. For execution, MilleniumDB demonstrates sub-0.01 second query latency for valid queries. This combined approach effectively addresses execution of the logical forms.

# 7 Future Work

There are various aspects of the project that can be improved and extended on in the future. Given the number of steps in the pipeline and other pre-pipeline steps, each step can be improved through some methods already discussed in the report.

## 7.1 Template Generation

Following the creation of the ontology and representative data, a key component of the pipeline involves generating question–query templates for each user profile. These templates are used throughout the system for validation, testing, and the generation of alternative test questions.

In this project, the initial template set was created using a GPT-based language model, with manual validation to ensure accuracy and representativeness. However, the current representative set is relatively limited in size, and it is unlikely that it fully captures the diversity of question types that a user profile may produce. As such, this set should be viewed as an initial foundation—one that is expected to grow and improve over time.

To support this growth, the system is designed to detect when a new user question significantly differs from existing templates (e.g., based on high embedding distance). In such cases, the system can prompt the LLM to generate a new question–query template pair. This proposed pair would then require manual validation—either by the user or a designated query engineer—to confirm its correctness and optimality. Upon approval, the new pair can be added to the representative set and vector store, thereby incrementally expanding the system's coverage and improving future retrieval performance.

## 7.2 Generalized templates

Although the current method of acquiring and using templates works very well another alternative that is more adaptable can also be considered. The system as it is takes templates with classes and predicates clearly identifiable and swaps in real values where applicable, however this system means that for every time a new structural modification is made to the database such as the addition of a new class, the existing template base is liable to break. As such taking these scalability concerns in mind another approach one can take to the templates is creating templates that are class agnostic. Which means that the question structure is such that any class can be fit into the question and the question would make sense. For example rather than having a question such as "How many patients have had a diagnosis of x?", where x denotes the variable that can be changed under the current structure. We would have a template that looks like "How many patients have had a y of x?, where y can represent any class. This type of template would ensure that any new classes are automatically accounted for and as such the test set of questions would never not cover the breadth of the dataset.

## 7.3 Contextualizing Relevant Ontology Components

Our initial approach incorporated a vector store to retrieve relevant ontology classes, similar to SPARQL-LLM [23]. This involved extracting noun phrases from the input question using the en_core_web_trf model from spaCy. Each noun phrase was embedded and compared to precomputed embeddings of ontology classes, where a match was accepted if the Euclidean distance between embeddings fell below a predefined similarity threshold. Ontology class embeddings were generated using both textual descriptions and the surrounding graph structure, as described in Section B.6.

While this method initially performed well, we observed that, following modifications to our dataset and the construction of the representative question set, most queries no longer included explicit references to ontology classes. As a result, this retrieval step offered limited practical value and therefore was removed from our final approach.

A potentially valuable direction for future work would be to apply a similar retrieval strategy for identifying relevant ontology predicates mentioned in the question, which are highly relevant for every query. This could involve encoding predicates into a vector store and matching them with extracted noun or maybe even verb phrases from the question. Such a method could provide more actionable context to guide the LLM in generating accurate and executable SPARQL queries. By retrieving relevant classes and predicates, one might be able to implement the generalized templates mentioned above.

## 7.4 Indexing Subgraphs

As described in section 3.1.1, a promising direction is improving SPARQL query performance for large datasets, based on the developed framework defining user intent across axes and the representative natural language question-SPARQL query dataset. Specifically, this could be achieved by pre-compiling simplified sub-graphs optimized to answer this a particular user profile's set of template SPARQL queries. Such an approach would allow querying smaller, profile-specific graphs, that is optimized for a specific user profile, resulting in fast query processing.

# References


[1] *Europe's digital decade: 2030 targets | European Commission*. en. URL: https://commission.europa.eu/strategy-and-policy/priorities-2019-2024/europe-fit-digital-age/europes-digital-decade-digital-targets-2030_en (visited on 03/25/2025).

[2] *European Health Data Space Regulation (EHDS) | European Commission*. en. URL: https://health.ec.europa.eu/ehealth-digital-health-and-care/european-health-data-space-regulation-ehds_en (visited on 03/25/2025).

[3] Tugce Schmitt et al. "What does it take to create a European Health Data Space? International commitments and national realities". en. In: *Zeitschrift für Evidenz, Fortbildung und Qualität im Gesundheitswesen* 179 (June 2023), pp. 1–7. ISSN: 18659217. DOI: 10.1016/j.zefq.2023.03.011. URL: https://linkinghub.elsevier.com/retrieve/pii/S1865921723000648 (visited on 03/26/2025).

[4] Shelly Sachdeva and Subhash Bhalla. "Using Knowledge Graph Structures for Semantic Interoperability in Electronic Health Records Data Exchanges". In: *Information* 13.2 (Jan. 2022), p. 52. ISSN: 2078-2489. DOI: 10.3390/info13020052. (Visited on 03/24/2025).

[5] Sola Shirai, Oshani Seneviratne, and Deborah L. McGuinness. *Applying Personal Knowledge Graphs to Health*. 2021. DOI: 10.48550/ARXIV.2104.07587. (Visited on 03/24/2025).

[6] *The Basics | National Institutes of Health (NIH)*. en. Apr. 2025. URL: https://www.nih.gov/health-information/nih-clinical-research-trials-you/basics (visited on 06/08/2025).

[7] Venkataramana Kandi and Sabitha Vadakedath. "Clinical Trials and Clinical Research: A Comprehensive Review". eng. In: *Cureus* 15.2 (Feb. 2023), e35077. ISSN: 2168-8184. DOI: 10.7759/cureus.35077.

[8] *ICH E6 Good clinical practice - Scientific guideline | European Medicines Agency (EMA)*. en. July 2002. URL: https://www.ema.europa.eu/en/ich-e6-good-clinical-practice-scientific-guideline (visited on 06/08/2025).

[9] Allison R. Baer et al. "Clinical Investigator Responsibilities". In: *Journal of Oncology Practice* 7.2 (Mar. 2011), pp. 124–128. ISSN: 1554-7477. DOI: 10.1200/JOP.2010.000216. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3051859/ (visited on 06/08/2025).

[10] Siamak Farshidi et al. "Understanding user intent modeling for conversational recommender systems: a systematic literature review". en. In: *User Modeling and User-Adapted Interaction* 34.5 (Nov. 2024), pp. 1643–1706. ISSN: 1573-1391. DOI: 10.1007/s11257-024-09398-x. URL: https://doi.org/10.1007/s11257-024-09398-x (visited on 06/08/2025).

[11] Xiang Ren et al. "Heterogeneous Graph-Based Intent Learning with Queries, Web Pages and Wikipedia Concepts". In: *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. WSDM 2014: Seventh ACM International Conference on Web Search and Data Mining. New York New York USA: ACM, Feb. 24, 2014, pp. 23–32. ISBN: 978-1-4503-2351-2. DOI: 10.1145/2556195.2556222. URL: https://dl.acm.org/doi/10.1145/2556195.2556222 (visited on 06/11/2025).

[12] Alon Talmor and Jonathan Berant. "The Web as a Knowledge-Base for Answering Complex Questions". In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 2018, pp. 641–651.

[13] Ping Wang, Tian Shi, and Chandan K. Reddy. *Text-to-SQL Generation for Question Answering on Electronic Medical Records*. 2019. DOI: 10.48550/ARXIV.1908.01839. (Visited on 03/24/2025).

[14] Junwoo Park et al. *Knowledge Graph-based Question Answering with Electronic Health Records*. 2020. DOI: 10.48550/ARXIV.2010.09394. (Visited on 03/24/2025).

[15] Anusri Pampari et al. "emrQA: A Large Corpus for Question Answering on Electronic Medical Records". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 2357–2368. DOI: 10.18653/v1/D18-1258. (Visited on 03/24/2025).

[16] Yanlin Feng, Simone Papicchio, and Sajjadur Rahman. *CypherBench: Towards Precise Retrieval over Full-scale Modern Knowledge Graphs in the LLM Era*. 2024. DOI: 10.48550/ARXIV.2412.18702. (Visited on 03/24/2025).

[17] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. "Semantics and complexity of SPARQL". In: *ACM Trans. Database Syst.* 34.3 (Sept. 2009). ISSN: 0362-5915. DOI: 10.1145/1567274.1567278. URL: https://doi.org/10.1145/1567274.1567278.

[18] Yushi Bai et al. "Longbench: A bilingual, multitask benchmark for long context understanding". In: *arXiv preprint arXiv:2308.14508* (2023).

[19] Mosh Levy, Alon Jacoby, and Yoav Goldberg. "Same task, more tokens: the impact of input length on the reasoning performance of large language models". In: *arXiv preprint arXiv:2402.14848* (2024).

[20] Jiaqi Li et al. "LooGLE: Can Long-Context Language Models Understand Long Contexts?" In: *arXiv preprint arXiv:2311.04939* (2023).

[21] Uri Shaham et al. "Zeroscrolls: A zero-shot benchmark for long text understanding". In: *arXiv preprint arXiv:2305.14196* (2023).

[22] Yushi Bai et al. "Answering complex logical queries on knowledge graphs via query computation tree optimization". In: *Proceedings of the 40th International Conference on Machine Learning*. ICML'23. Honolulu, Hawaii, USA: JMLR.org, 2023.

[23] Vincent Emonet et al. *LLM-based SPARQL Query Generation from Natural Language over Federated Knowledge Graphs*. 2025. arXiv: 2410.06062 [cs.DB]. URL: https://arxiv.org/abs/2410.06062.

[24] Erik Arakelyan et al. "Complex Query Answering with Neural Link Predictors". In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=Mos9F9kDwkz.

[25] Wilco v. Leeuwen et al. "AvantGraph query processing engine". In: *Proc. VLDB Endow.* 15.12 (Aug. 2022), 3698–3701. ISSN: 2150-8097. DOI: 10.14778/3554821.3554878. URL: https://doi.org/10.14778/3554821.3554878.

[26] Wilco van Leeuwen, George Fletcher, and Nikolay Yakovets. "A General Cardinality Estimation Framework for Subgraph Matching in Property Graphs". In: *IEEE Transactions on Knowledge and Data Engineering* (2022).

[27] J. Patel et al. "Quickstep: A Data Platform Based on the Scaling-Up Approach". In: *Proceedings of the VLDB Endowment* 11.6 (2018), pp. 663–676.

[28] Angela Bonifati et al. *Querying Graphs*. Vol. 10. Synthesis Lectures on Data Management 3. Morgan & Claypool Publishers, 2018, pp. 1–184.

[29] Diego Arroyuelo et al. "Worst-case optimal graph joins in almost no space". In: *Proceedings of the 2021 International Conference on Management of Data*. 2021.

[30] Domagoj Vrgoč et al. "MillenniumDB: A Persistent, Open-Source, Graph Database". In: *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*. 2021.

[31] Hannah Bast and Björn Buchhold. "QLever: A Query Engine for Efficient SPARQL+Text Search". In: *Proceedings of the 2017 ACM International Conference on Information and Knowledge Management (CIKM)*. Singapore, 2017. DOI: 10.1145/3132847.3132921.

[32] Vincent Emonet et al. *LLM-based SPARQL Query Generation from Natural Language over Federated Knowledge Graphs*. 2024. DOI: 10.48550/ARXIV.2410.06062. (Visited on 03/24/2025).

[33] Weijie Chen et al. *KG-Retriever: Efficient Knowledge Indexing for Retrieval-Augmented Large Language Models*. 2024. DOI: 10.48550/ARXIV.2412.05547. (Visited on 03/24/2025).

[34] URL: https://digirepo.nlm.nih.gov/master/borndig/9918249008406676/9918249008406676.pdf (visited on 06/12/2025).

[35] *How to Edit Your Study Record | ClinicalTrials.gov*. URL: https://clinicaltrials.gov/submit-studies/prs-help/how-edit-record (visited on 06/12/2025).

# A Representative question-query templates

## A.1 User Profiles

| Trial Phase | User Role | Intent Category | Example Question |
|---|---|---|---|
| Planning & Design | Principal Investigator | Cohort Size Estimation | How many patients have metastatic breast cancer under age 50? |
| Planning & Design | Clinical Researcher | Eligibility Criteria Specification | Can we find patients with liver disease but no renal complications? |
| Planning & Design | Data Analyst | Population Characteristics Summary | What's the age distribution of hypertensive patients? |
| Planning & Design | Biostatistician | Inclusion/Exclusion Criteria Tuning | How many patients would be excluded due to comorbidity X? |
| Planning & Design | Regulatory Affairs | Protocol Compliance Preview | Do the inclusion criteria align with EMA/ICH guidelines? |
| Planning & Design | Trial Designer | Comparator Arm Feasibility | Do we have enough patients not receiving Drug A for a control arm? |
| Recruitment & Screening | Site Coordinator | Patient Eligibility Verification | Is patient 143462 eligible based on age and previous diagnosis? |
| Recruitment & Screening | Clinical Researcher | Recruitment Funnel Tracking | How many patients passed screening this month? |
| Recruitment & Screening | Data Analyst | Eligibility Rule Simulation | What if we drop the age criterion? How many new patients qualify? |
| Recruitment & Screening | Recruiter | Candidate Listing | List patients with HIV diagnosis and no hepatitis B coinfection. |
| Intervention | Clinical Monitor | Protocol Adherence Monitoring | Who missed their scheduled drug administration last week? |
| Intervention | Pharmacovigilance Officer | Adverse Event Tracking | Show patients who experienced nausea after Drug B. |
| Intervention | Trial Nurse | Visit Schedule Compliance | Which patients didn't attend their second trial visit? |
| Intervention | Clinical Investigator | Unblinding Requests / AE Reporting | Is there a higher dropout rate in the treatment group? |
| Intervention | Biostatistician | Interim Analysis Preparation | What's the preliminary efficacy trend of Drug C? |
| Follow-up & Analysis | Clinical Data Manager | Missing Data Identification | Which patients have incomplete lab results in week 8? |
| Follow-up & Analysis | Statistician | Outcome Analysis | Compare mortality rates between Drug A and placebo groups. |
| Follow-up & Analysis | Trial Sponsor | Efficacy Signal Detection | Is there a significant effect of Drug D on blood pressure after 3 months? |
| Follow-up & Analysis | Quality Assurance Auditor | Data Consistency Checking | Are there discrepancies between recorded adverse events and lab findings? |
| Follow-up & Analysis | Health Economist / Market Access | Cost-Effectiveness Cohort Exploration | How many patients meet NICE's criteria for cost-effectiveness of Drug E? |
| Reporting & Publication | Clinical Researcher | Patient Outcome Reporting | List patients with complete response after 12 weeks of therapy. |
| Reporting & Publication | Trial Sponsor / Medical Writer | Data Summary for Publication | Summarize key safety and efficacy indicators by arm. |

**Table 2 – continued from previous page**

| Trial Phase | User Role | Intent Category | Example Question |
|---|---|---|---|
| Reporting & Publication | Regulatory Affairs | Submission Package Validation | Which patients had serious adverse events requiring reporting to the regulator? |

Table 2: User Profiles and their Intents along with some examples

## A.2 Question-Query Templates for Recruitment phase, Clinical Researcher role and Eligibility Screening intent in Clinical Trial

| Count | Question Template |
|---|---|
| 0 | What is the number of patients diagnosed with §Diagnosis§? |

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
PREFIX icd: <https://www.biomedit.ch/rdf/sphn-schema/sphn/icd#>
SELECT (COUNT(*) AS ?numPatients) WHERE {
SELECT DISTINCT ?patient WHERE {
?patient sphn:hasDiagnosis ?diag .
?diag sphn:hasCode ?code .
?code sphn:hasCodeValue icd:§Diagnosis§ .
}
}
```

| 1 | Which patients have received a diagnosis of §Diagnosis§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
PREFIX icd: <https://www.biomedit.ch/rdf/sphn-schema/sphn/icd#>
SELECT ?patient WHERE {
?patient sphn:hasDiagnosis ?diag .
?diag sphn:hasCode ?code .
?code sphn:hasCodeValue icd:§Diagnosis§ .
}
```

| 2 | What patients have not met the §Threshold§ for their laboratory tests? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?event sphn:hasSubjectPseudoIdentifier ?patient .
?event sphn:hasLabResult ?res .
?res sphn:hasQuantityValue ?val .
FILTER(?val < §Threshold§)
}
```

| 3 | Which patients are older than §Age§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?patient sphn:hasBirthDate ?dob .
BIND(year(NOW()) - year(?dob) AS ?age)
FILTER(?age >= §Age§)
}
```

| 4 | Which individuals have had exposure to the drug §Drug§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?presc sphn:hasSubjectPseudoIdentifier ?patient .
?presc sphn:hasDrug ?drug .
?drug sphn:hasCode ?code .
?code sphn:hasValue "§Drug§"
}
```

| 5 | Which patient's laboratory results have exceeded the §Threshold§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?event sphn:hasSubjectPseudoIdentifier ?patient .
?event sphn:hasLabResult ?res .
?res sphn:hasQuantityValue ?val .
FILTER(?val> §Threshold§)
}
```

| 6 | What patients have received §Drug§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?presc sphn:hasSubjectPseudoIdentifier ?patient .
?presc sphn:hasDrug ?drug .
?drug sphn:hasCode ?code .
?code sphn:hasValue "§Drug§
}
```

| 7 | Which patients have experienced an adverse event classified as §Severity§? |
|---|---|

```
PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/>
SELECT ?patient WHERE {
?ae <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
sphn:AdverseEvent .
?ae sphn:hasSubjectPseudoIdentifier ?patient .
?ae sphn:hasSeverity ?sev .
FILTER(?sev IN ("§Severity§"))
}
```

Table 3: Examples of Templates and Queries

# B Logical Form Generation

## B.1 Ontology

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

#################################################################
# Classes
#################################################################

sphn:AdverseEvent a rdfs:Class,
        owl:Class ;
    rdfs:label "AdverseEvent" ;
    rdfs:comment "Auto-generated comment for class https://www.biomedit.
        ch/rdf/sphn-schema/sphn/AdverseEvent" .

sphn:Code a rdfs:Class,
        owl:Class ;
    rdfs:label "Code" ;
    rdfs:comment "Symbols and/or expressions defined in a coding system.
        " .

sphn:Diagnosis a rdfs:Class,
        owl:Class ;
    rdfs:label "Diagnosis" ;
    rdfs:comment "Determination of the presence of a disease, condition,
         or injury from expressed signs and symptoms and assessments
        such as physical examination, laboratory test, or the like." .

sphn:Drug a rdfs:Class,
        owl:Class ;
    rdfs:label "Drug" ;
    rdfs:comment "Any substance with the intent to prevent, diagnose,
        treat, or relieve symptoms of a disease or abnormal condition."
        .

sphn:DrugPrescription a rdfs:Class,
        owl:Class ;
    rdfs:label "Drug Prescription" ;
    rdfs:comment "Plan that defines at which frequency a drug should be
        administered to a patient with a given quantity; at every
        frequency time point a drug administration event should occur."
        .

sphn:FollowUpVisit a rdfs:Class,
        owl:Class ;
    rdfs:label "FollowUpVisit" ;
    rdfs:comment "Auto-generated comment for class https://www.biomedit.
        ch/rdf/sphn-schema/sphn/FollowUpVisit" .

sphn:LabResult a rdfs:Class,
        owl:Class ;
    rdfs:label "Lab Result" ;
    rdfs:comment "Outcome, value, or information which gives insight
        about a laboratory test." .
```

```
sphn:LabTestEvent a rdfs:Class,
        owl:Class ;
    rdfs:label "Lab Test Event" ;
    rdfs:comment "Occurrence in which one or multiple laboratory tests
        are performed on a biological specimen at a given time." .

sphn:MedicalProcedure a rdfs:Class,
        owl:Class ;
    rdfs:label "Medical Procedure" ;
    rdfs:comment "Invasive or non-invasive intervention performed for,
        with or on behalf of an individual whose purpose is to assess,
        improve, maintain, promote or modify health, functioning or
        health conditions." .

sphn:Patient a rdfs:Class,
        owl:Class ;
    rdfs:label "Patient" ;
    rdfs:comment "Auto-generated comment for class https://www.biomedit.
        ch/rdf/sphn-schema/sphn/Patient" .

sphn:Substance a rdfs:Class,
        owl:Class ;
    rdfs:label "Substance" ;
    rdfs:comment "Any matter of defined composition that has discrete
        existence, whose origin may be biological, mineral or chemical."
         .

sphn:SubjectPseudoIdentifier a owl:Class ;
    rdfs:label "Patient Identifier" ;
    rdfs:comment "A coded unique identifier assigned by a data provider
        for a specific project to conceal the identity of an individual.
        " .

#################################################################
# Object Properties
#################################################################

<http://kg-representation-ehr.org/ontology/clinicalIntent#
    hasDrugPrescription> a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "clinicalIntent#hasDrugPrescription" ;
    rdfs:comment "Auto-generated comment for property http://kg-
        representation-ehr.org/ontology/clinicalIntent#
        hasDrugPrescription" ;
    rdfs:domain sphn:Patient ;
    rdfs:range sphn:DrugPrescription .

<http://kg-representation-ehr.org/ontology/clinicalIntent#
    hasMedicalProcedure> a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "clinicalIntent#hasMedicalProcedure" ;
    rdfs:comment "Auto-generated comment for property http://kg-
        representation-ehr.org/ontology/clinicalIntent#
        hasMedicalProcedure" ;
    rdfs:domain sphn:Patient ;
    rdfs:range sphn:MedicalProcedure .

sphn:hasActiveIngredient a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "has Active Ingredient" ;
    rdfs:comment "Active component of the concept." ;
    rdfs:domain sphn:Drug ;
    rdfs:range sphn:Substance .
```

```
sphn:hasCode a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "has Code" ;
    rdfs:comment "Coded information specifying the concept." ;
    rdfs:domain sphn:Diagnosis,
        sphn:LabResult,
        sphn:MedicalProcedure,
        sphn:Substance ;
    rdfs:range sphn:Code .

sphn:hasDiagnosis a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "hasDiagnosis" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasDiagnosis" ;
    rdfs:domain sphn:Patient ;
    rdfs:range sphn:Diagnosis .

sphn:hasDrug a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "has Drug" ;
    rdfs:comment "Medication associated to the concept." ;
    rdfs:domain sphn:DrugPrescription ;
    rdfs:range sphn:Drug .

sphn:hasLabTest a owl:ObjectProperty ;
    rdfs:label "has Lab Test" ;
    rdfs:comment "Lab test associated to the concept." ;
    rdfs:domain sphn:LabTestEvent ;
    rdfs:range sphn:LabResult .

sphn:hasSubjectPseudoIdentifier a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "has Subject Pseudo Identifier" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasSubjectPseudoIdentifier" ;
    rdfs:domain sphn:Diagnosis,
        sphn:DrugPrescription,
        sphn:LabTestEvent,
        sphn:MedicalProcedure ;
    rdfs:range sphn:SubjectPseudoIdentifier .

sphn:hasLabResult a rdf:Property,
        owl:ObjectProperty ;
    rdfs:label "hasLabResult" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasLabResult" .
    rdfs:domain sphn:LabTestEvent ;
    rdfs:range sphn:LabResult .

#################################################################
# Datatype Properties
#################################################################

sphn:hasCodingSystemAndVersion a owl:DatatypeProperty ;
    rdfs:label "has Coding System and Version" ;
    rdfs:comment "Name and version of the coding system." ;
    rdfs:domain sphn:Code ;
    rdfs:range xsd:string .
```

```
sphn:hasValue a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasValue" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasValue" .
    rdfs:domain sphn:Code ;
    rdfs:range xsd:string .

sphn:hasVisitDate a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasVisitDate" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasVisitDate" ;
    rdfs:domain sphn:FollowUpVisit ;
    rdfs:range xsd:date .

sphn:hasQuantityValue a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasQuantityValue" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasQuantityValue" ;
    rdfs:domain sphn:LabResult ;
    rdfs:range xsd:float .

sphn:hasSeverity a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasSeverity" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasSeverity" ;
    rdfs:domain sphn:AdverseEvent ;
    rdfs:range xsd:string .

sphn:hasCodeValue a rdf:Property ;
    rdfs:label "hasCodeValue" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasCodeValue" ;
    rdfs:domain sphn:Code ;
    rdfs:range <https://www.biomedit.ch/rdf/sphn-schema/sphn/icd#...> .

sphn:hasEventDate a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasEventDate" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasEventDate" ;
    rdfs:domain sphn:LabTestEvent,
        sphn:AdverseEvent ;
    rdfs:range xsd:date .

sphn:hasBirthDate a rdf:Property,
        owl:DatatypeProperty ;
    rdfs:label "hasBirthDate" ;
    rdfs:comment "Auto-generated comment for property https://www.
        biomedit.ch/rdf/sphn-schema/sphn/hasBirthDate" ;
    rdfs:domain sphn:Patient ;
    rdfs:range xsd:date .
```

Figure 6: Ontology of the augmented dataset

## B.2 Baseline Prompt

```
INITIAL_PROMPT =
    Task: Generate a SPARQL SELECT statement for querying a graph database.
    Question: Which patients have been diagnosed with I427?
    Answer
PREFIX sphn: https://www.biomedit.ch/rdf/sphn-schema/sphn/
PREFIX icd: https://www.biomedit.ch/rdf/sphn-schema/sphn/icd

SELECT ?patient WHERE {{
  ?patient sphn:hasDiagnosis ?diag .
  ?diag sphn:hasCode ?code .
  ?code sphn:hasCodeValue
         icd:I427 .
}}

Keep in mind that you might need several classes in order to provide the correct answer.

Instructions:
Use only the node types and properties provided in the ontology.
Do not use any node types and properties that are not explicitly provided.
Include all necessary prefixes and relations.

The ontology is:
{ontology}

Note: Be as concise as possible.
Do not include any explanations or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except for the SPARQL query generated.

The question is:{question}
```

Figure 7: Baseline Prompt Template with initial example query

## B.3 Contextualized prompt

```
Task: Generate a SPARQL SELECT statement for querying a graph database.

Instructions:
The SPARQL SELECT statement should answer the user question by filling
    in the provided query templates.
Use the ontology terminology to ensure the query is executable on the
    graph database.
Use the question mentioned ontology classes to guide the selection of
    the appropriate query template.

--- Ontology ---
{ontology}

--- Templates ---
{query_templates}

--- Examples ---
{examples}

--- Notes ---
Be as concise as possible.
Do not include any explanations or apologies in your responses.
Do not include any text except for the SPARQL query generated.

--- User Question ---
The user question is:
{question}

The question mentioned ontology classes are:
{relevant_classes}

The answered SPARQL query is:
```

Figure 8: Prompt template to add context

## B.4 Query validation prompt

```
Task: Fix a SPARQL SELECT statement for querying a graph database that returned an error.
This query is supposed to answer some question, but it caused an error when executed.
Instructions:
Fix ONLY the error that is provided. Do not significantly change the structure of the query, but fix it to be valid.
The query should still answer the original question.

The ontology is:
{ontology}

Note: Be as concise as possible.
Do not include any explanations or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except for the SPARQL query generated.

Here is the SPARQL query that caused an error:
{query}

Here is the error message:
{error}
```

Figure 9: Prompt template to correct a query that had incorrect SPARQL syntax

## B.5 Test set generation prompt

```
You are a paraphrasing assistant. Each time you receive an input question template, you must output exactly one paraphrased question that:
1. Keeps the same meaning and intent as the original.
2. Remains a valid question (ends with a question mark).
3. Preserves all placeholder tokens (e.g. §Threshold§, §Diagnosis§) exactly do not rename, alter, or remove them, but you may move them within the sentence.
4. Uses different wording and sentence structure from the original.

Respond only with the paraphrased question do not include any commentary or extra text.
```

Figure 10: Prompt provided to shuffle question structure

```
# SPARQL template
sparql_template = """SELECT ?patient
WHERE {
    ?patient sphn:hasDiagnosis ?diag .
    ?diag     sphn:hasCode      ?code .
    ?code    sphn:hasCodeValue §Diagnosis§ .
    ?event   sphn:hasSubjectPseudoIdentifier ?patient .
    ?event   sphn:hasLabResult               ?res .
    ?res     sphn:hasQuantityValue           ?val .
    FILTER(?val > §Threshold§)
}
"""

# Natural-language question template
question_template = (
    "Which patients have been diagnosed with §Diagnosis§ "
    "and have had a lab test surpass the value §Threshold§?"
)
```

Figure 11: Example input provided to the system

## B.6 Ontology Class Representation Prompt

```
**Class:** ...

**Definition:**
...

**Relations:**
(format: Class -> Relation -> Class)
- ...
```

Figure 12: How a class is represented in the ontology

## B.7 Example Query and Template

| Question | Which patients have received a diagnosis of I10? |
|---|---|
| SparQL Template | PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/><br>PREFIX icd: <https://www.biomedit.ch/rdf/sphn-schema/sphn/icd#><br>SELECT ?patient WHERE {<br>?patient sphn:hasDiagnosis ?diag .<br>?diag sphn:hasCode ?code .<br>?code sphn:hasCodeValue icd:§Diagnosis§ .<br>} |
| SparQL Generated | PREFIX sphn: <https://www.biomedit.ch/rdf/sphn-schema/sphn/><br>PREFIX icd: <https://www.biomedit.ch/rdf/sphn-schema/sphn/icd#><br>SELECT ?patient WHERE {<br>?patient sphn:hasDiagnosis ?diag .<br>?diag sphn:hasCode ?code .<br>?code sphn:hasCodeValue icd:I10 .<br>} |

Table 4: An example question-query template and a query it generates